Complete Workshop & Tool Specification: Personalized Worked Example Generator

Building AI-Powered Educational Tools Grounded in Cognitive Load Theory

---

Table of Contents

Workshop Overview

Title

Building Personalized Worked Example Generators with AI
Demonstrating Cognitive Load Theory Principles through PydanticAI

Duration

3 hours

Target Audience

- Educators interested in AI tools
- Instructional designers
- Learning technologists
- Basic Python knowledge required (lists, dictionaries, functions)
- No statistics background required

Learning Objectives

By the end of this workshop, participants will:

1. **Understand** the worked example effect and personalization principle from Cognitive Load Theory
2. **Build** a working personalized example generator using PydanticAI
3. **Deploy** an interactive demo to HuggingFace Spaces
4. **Possess** a template for creating domain-specific educational tools

Pedagogical Foundation

This workshop is grounded in **Cognitive Load Theory** (Sweller, 1988), specifically:

The Worked Example Effect:

> "A 'worked example' is a problem that has already been solved for the learner, with every step fully explained and clearly shown. The 'worked example effect' is the widely replicated finding that novice learners who are given worked examples to study perform better on subsequent tests than learners who are required to solve the equivalent problems themselves." (NSW CESE, 2017, p. 7)

Why It Works:

- Unguided problem-solving overloads working memory
- Studying worked examples reduces cognitive load
- More working memory capacity available for schema construction
- Better learning outcomes for novices

Our Addition - Personalization:

- Familiar contexts reduce extraneous cognitive load
- Personal relevance increases germane load (motivation)
- Better schema formation and transfer

Workshop Design

Materials Required

For Participants (Pre-Workshop):

- [ ] Python 3.10 or higher installed
- [ ] OpenAI API key (from platform.openai.com)
- [ ] HuggingFace account (from huggingface.co)
- [ ] Code editor or IDE

Installation Commands:

```bash
pip install marimo pydantic-ai openai
export OPENAI_API_KEY="sk-your-key-here"
```

Test Installation:

```bash
marimo tutorial intro
```

Part 1: Introduction & Setup (30 minutes)

Activity 1.1: Welcome & Motivation (10 min)

Instructor Script:

```
Welcome! Today we're building an AI tool that demonstrates a powerful principle
from Cognitive Load Theory: the worked example effect.

[Show slide comparing examples]

Question: Which would be easier to learn from?

Example A: "Calculate the correlation between Variable X and Variable Y"
Example B: "Calculate the correlation between your weekly running distance
        and your 5K race times"

Most people find personalized examples easier. Why?

Because familiar contexts require less mental effort to process. That's
cognitive load theory in action.

Today we'll build a tool that automatically creates personalized worked
examples for learners in any domain.
```

Slides to Show:

Slide 1: The Worked Example Effect

```
TRADITIONAL APPROACH (High Cognitive Load):
├─ Give student unsolved problem
├─ Student struggles independently
└─ May succeed but doesn't learn the pattern

WORKED EXAMPLE APPROACH (Lower Cognitive Load):
├─ Show student solved problem with explanations
├─ Student studies the solution
└─ Better learns the transferable pattern

Research: Effect size of 0.52 (Crissman, 2006)
```

Slide 2: Why Personalization Matters

Generic Example:
"Calculate mean and SD for: [1, 2, 3, 4, 5]"
↓
High extraneous load (abstract context)

Personalized Example:
"Calculate mean and SD for your last 5 race times: [22, 21, 19, 20, 18] minutes"
↓
Low extraneous load (familiar context)
More working memory for learning!

Activity 1.2: Environment Setup (15 min)

Live Demonstration:

```python
# Test that everything works
import marimo as mo

mo.md("# Test Installation")

# Test cell
print("If you see this, marimo works!")

# Save as test.py and run
# marimo edit test.py
```

Troubleshooting Checklist:

- Python version: `python --version` (need 3.10+)
- Marimo installed: `marimo --version`
- API key set: `echo $OPENAI_API_KEY`
- If issues on Windows: Use `set OPENAI_API_KEY=...` instead of `export`

Activity 1.3: PydanticAI Quick Demo (5 min)

Live Coding:

```python
```

```python
from pydantic import BaseModel
from pydantic_ai import Agent

class Greeting(BaseModel):
    message: str
    enthusiasm_level: int

agent = Agent('openai:gpt-4o', result_type=Greeting)
result = agent.run_sync("Say hello enthusiastically!")

print(f"Message: {result.data.message}")
print(f"Enthusiasm: {result.data.enthusiasm_level}")
```

Key Points:

- **Structured outputs** - not just text
- **Type safety** with Pydantic
- **Sync vs async** - we'll use async in notebooks

Part 2: Understanding the Problem Domain (30 minutes)

Activity 2.1: Three Learning Domains (10 min)

Instructor Leads Discussion:

We'll support three domains today:

1. PROGRAMMING (Python for beginners)
   - Concepts: for loops, functions, list comprehensions
   - Context: Personal projects, hobbies, interests
   - Example: "Generate navigation links for your photography portfolio"

2. HEALTH SCIENCES (Statistics for undergrads)
   - Concepts: correlation, mean/SD, t-tests, confidence intervals
   - Context: Sports, fitness, nutrition, personal health
   - Example: "Analyze correlation between your training volume and race times"

3. AGRONOMY (Agricultural science)
   - Concepts: yield prediction, fertilizer optimization, cost-benefit
   - Context: Specific crops, family farms, regional agriculture
   - Example: "Optimize nitrogen application for your coffee farm"

Question: What makes a good personalizable example?
[Collect answers]

Key criteria:
✓ Uses learner's actual interests
✓ Realistic data in their context
✓ Clear connection to their goals
✓ Familiar terminology

Activity 2.2: Design Learner Profile Structure (15 min)

Collaborative Live Coding:

python

```python
from pydantic import BaseModel, Field
from typing import Literal

# Start simple, build up together
class LearnerProfile(BaseModel):
    name: str

    # What else do we need? [Ask participants]
```

Build This Together:

python

```python
class LearnerProfile(BaseModel):
    """Collect learner information for personalization"""

    name: str = Field(
        description="Learner's first name"
    )

    domain: Literal["programming", "health_sciences", "agronomy"] = Field(
        description="Learning domain"
    )

    specific_interest: str = Field(
        description="Specific interest within domain",
        examples=["web development", "sports nutrition", "coffee farming"]
    )

    hobby_or_passion: str = Field(
        description="A hobby or passion they have",
        examples=["photography", "cycling", "sustainable farming"]
    )

    goal: str = Field(
        description="What they want to achieve",
        examples=["build portfolio site", "improve performance", "increase yield"]
    )

    background_level: Literal["beginner", "intermediate", "advanced"] = Field(
        description="Their current level"
    )
```

Test It:

```python
# Create example profile together
sarah = LearnerProfile(
    name="Sarah",
    domain="programming",
    specific_interest="web development",
    hobby_or_passion="baking",
    goal="build a recipe sharing website",
    background_level="beginner"
)

print(sarah.model_dump_json(indent=2))
```

Activity 2.3: Define Worked Example Structure (5 min)

Guided Discussion:

What should a complete worked example include?
[Collect on whiteboard/screen]

- Title? YES
- Problem statement? YES
- Given data? YES
- Solution steps? YES
- Final answer? YES
- Anything else?
  - Connection to their goal? YES!
  - Practice suggestion? YES!

Code Together:

python

```python
class PersonalizedWorkedExample(BaseModel):
    """A complete worked example tailored to the learner"""

    title: str = Field(
        description="Engaging title incorporating learner's interest"
    )

    problem_statement: str = Field(
        description="Problem framed in learner's context (3-4 sentences)"
    )

    given_data: str = Field(
        description="Data in familiar context (can include code/tables)"
    )

    step_by_step_solution: list[str] = Field(
        description="Clear steps with explanations"
    )

    final_answer: str = Field(
        description="Answer with interpretation relevant to learner"
    )

    connection_to_goal: str = Field(
        description="How this relates to their goal (2-3 sentences)"
    )

    practice_suggestion: str = Field(
        description="Similar problem they could try next"
    )
```

Part 3: Building the Generator (50 minutes)

Activity 3.1: Create Concept Library (10 min)

Explain:

We need a library of concepts for each domain.
These define WHAT we can teach.

Code Together:

python

```
CONCEPTS = {
    "programming": [
        {
            "name": "For Loops",
            "abstract": "Iterate through a sequence of items",
            "difficulty": "beginner",
            "typical_use": "Processing lists, repeating actions"
        },
        {
            "name": "List Comprehensions",
            "abstract": "Create lists using concise syntax",
            "difficulty": "intermediate",
            "typical_use": "Transform data, filter lists elegantly"
        },
        {
            "name": "Functions with Parameters",
            "abstract": "Create reusable code blocks that accept inputs",
            "difficulty": "beginner",
            "typical_use": "Organize code, avoid repetition"
        }
    ],

    "health_sciences": [
        {
            "name": "Correlation Analysis",
            "abstract": "Measure relationship between two variables",
            "difficulty": "intermediate",
            "typical_use": "Find relationships in health data"
        },
        {
            "name": "Mean and Standard Deviation",
            "abstract": "Describe central tendency and variability",
            "difficulty": "beginner",
            "typical_use": "Summarize health measurements"
        },
        {
            "name": "Independent T-Test",
            "abstract": "Compare means between two groups",
            "difficulty": "intermediate",
            "typical_use": "Test intervention effectiveness"
        }
    ],

    "agronomy": [
        {
            "name": "Yield Prediction",
```

```
      "abstract": "Estimate crop output from inputs",
      "difficulty": "intermediate",
      "typical_use": "Plan harvest, resource allocation"
    },
    {
      "name": "NPK Optimization",
      "abstract": "Calculate optimal fertilizer ratios",
      "difficulty": "intermediate",
      "typical_use": "Maximize yield, minimize cost"
    },
    {
      "name": "Growing Degree Days",
      "abstract": "Calculate heat accumulation for crops",
      "difficulty": "beginner",
      "typical_use": "Predict crop development stages"
    }
  ]
}
```

Optional: If time permits, have participants add one concept per domain group.

Activity 3.2: Build the AI Agent (20 min)

Critical Discussion:

Now we create the AI agent that generates examples.
What instructions should we give it?

Key requirements:
- Weave in learner's interests naturally
- Use their name throughout
- Make data realistic
- Match their level
- Connect to their goal

Live Code:

python

```
from pydantic_ai import Agent

example_generator = Agent(
    'openai:gpt-4o',
    result_type=PersonalizedWorkedExample,
    system_prompt="""You are an expert educator who creates highly personalized
    worked examples that connect abstract concepts to learners' lived experiences.

    CRITICAL INSTRUCTIONS:
    1. Weave the learner's interests, hobbies, and goals naturally into the example
    2. Use their name throughout to increase personal connection
    3. Make data and scenarios feel authentic to their context
    4. Keep explanations clear but connect to what they care about
    5. The example should feel like it was written specifically for this person
    6. Match complexity to their level (beginner/intermediate/advanced)
    7. Make the connection to their goal explicit and motivating
    8. Use concrete numbers and realistic data
    9. For programming: Include complete, runnable code with comments
    10. For quantitative problems: Show every calculation step explicitly

    STRUCTURE YOUR EXAMPLES:
    - Start with engaging title mentioning their interest
    - Frame problem in their context (use their name and interests)
    - Present data that feels real to their situation
    - Walk through steps clearly with explanations
    - For code: include comments explaining each part
    - For math: show each calculation explicitly
    - Connect final answer to their goal
    - Suggest related practice problem in their context

    AVOID:
    - Generic examples with personal details tacked on
    - Forced or artificial connections
    - Too much jargon for beginners
    - Abstract variable names (use meaningful names from context)
    - Skipping steps in solutions
    """
)
```

Explain Each Part:

- `result_type` ensures structured output
- System prompt is crucial for quality
- Specific instructions > general instructions

- Balance detail with clarity

Activity 3.3: Create Generation Function (10 min)

```python
async def generate_personalized_example(
    profile: LearnerProfile,
    concept: dict
) -> PersonalizedWorkedExample:
    """Generate a personalized worked example"""

    prompt = f"""
    Create a worked example for:

    LEARNER PROFILE:
    - Name: {profile.name}
    - Domain: {profile.domain}
    - Specific interest: {profile.specific_interest}
    - Hobby/passion: {profile.hobby_or_passion}
    - Goal: {profile.goal}
    - Level: {profile.background_level}

    CONCEPT TO TEACH:
    - Name: {concept['name']}
    - Abstract description: {concept['abstract']}
    - Difficulty: {concept['difficulty']}
    - Typical use: {concept['typical_use']}

    Create a worked example that teaches this concept using {profile.name}'s
    specific context. The example should feel personal and relevant to their
    goal of "{profile.goal}".

    Make the problem realistic and the data believable for their situation.
    """

    result = await example_generator.run(prompt)
    return result.data
```

Activity 3.4: Test the Generator (10 min)

Live Demo:

```python
```

```python
# Test with Sarah's profile
test_concept = CONCEPTS["programming"][0]  # For Loops

print("Generating example for Sarah...")
print(f"Concept: {test_concept['name']}")
print()

example = await generate_personalized_example(sarah, test_concept)

print(f"Title: {example.title}")
print(f"\nProblem: {example.problem_statement[:100]}...")
print(f"\nSteps: {len(example.step_by_step_solution)} steps")
print(f"\nConnection to goal: {example.connection_to_goal[:80]}...")
```

Group Discussion:

- Is it personalized?
- Is it pedagogically sound?
- What could be improved?

Part 4: Building the Interactive Interface (40 minutes)

Activity 4.1: Marimo Basics (5 min)

Quick Overview:

```python
import marimo as mo

# Marimo is reactive - cells auto-update when dependencies change

# Create a text input
name = mo.ui.text(label="Your name:")
name  # Display it

# Use its value
if name.value:
    mo.md(f"Hello, {name.value}!")
```

Key Concepts:

- Cells are reactive
- UI elements have `.value` property
- Display with just the variable name

- `mo.md()` for markdown formatting

Activity 4.2: Build Profile Input Form (15 min)

Code Together:

```python
```

```
# Create all input widgets
name_input = mo.ui.text(
    label="Your first name:",
    placeholder="e.g., Maria"
)

domain_input = mo.ui.dropdown(
    label="Choose your learning domain:",
    options={
        "Programming (Python)": "programming",
        "Health Sciences (Statistics)": "health_sciences",
        "Agronomy (Agricultural Science)": "agronomy"
    }
)

interest_input = mo.ui.text(
    label="Your specific interest:",
    placeholder="e.g., web development, sports nutrition, coffee farming"
)

hobby_input = mo.ui.text(
    label="A hobby or passion:",
    placeholder="e.g., photography, cycling, cooking"
)

goal_input = mo.ui.text(
    label="What you want to achieve:",
    placeholder="e.g., build portfolio site, improve performance, increase yield"
)

level_input = mo.ui.dropdown(
    label="Your current level:",
    options=["beginner", "intermediate", "advanced"]
)

# Display form
mo.vstack([
    mo.md("## 👤 Tell Us About Yourself"),
    name_input,
    domain_input,
    interest_input,
    hobby_input,
    goal_input,
    level_input
])
```

Activity 4.3: Add Dynamic Concept Selection (10 min)

python

```
# Check if profile is complete
profile_complete = all([
    name_input.value,
    domain_input.value,
    interest_input.value,
    hobby_input.value,
    goal_input.value,
    level_input.value
])

# Create profile if complete
if profile_complete:
    learner_profile = LearnerProfile(
        name=name_input.value,
        domain=domain_input.value,
        specific_interest=interest_input.value,
        hobby_or_passion=hobby_input.value,
        goal=goal_input.value,
        background_level=level_input.value
    )

    mo.callout(
        f"✅ Profile Complete! Ready to generate examples, {learner_profile.name}.",
        kind="success"
    )

    # Show concept selector
    available_concepts = CONCEPTS[learner_profile.domain]

    concept_selector = mo.ui.dropdown(
        label="Choose a concept to learn:",
        options={
            f"{c['name']} ({c['difficulty']})": c
            for c in available_concepts
        }
    )

    mo.vstack([
        mo.md("## 📚 Choose a Concept"),
        concept_selector
    ])
```

Activity 4.4: Generate and Display Example (10 min)

python

```python
# Generate button
if concept_selector and concept_selector.value:
    generate_button = mo.ui.button(
        label="✨ Generate My Personalized Example",
        kind="success"
    )

    generate_button

# Generate when clicked
if generate_button and generate_button.value:

    with mo.status.spinner(title="Creating your example... (30-60 seconds)"):
        example = await generate_personalized_example(
            profile=learner_profile,
            concept=concept_selector.value
        )

    # Display formatted
    mo.vstack([
        mo.md(f"# {example.title}"),
        mo.md("## 📋 The Problem"),
        mo.md(example.problem_statement),
        mo.md("### Given Data"),
        mo.md(example.given_data),
        mo.md("---"),
        mo.md("## 💡 Step-by-Step Solution"),
        *[mo.md(f"**Step {i}:**\n\n{step}")
          for i, step in enumerate(example.step_by_step_solution, 1)],
        mo.md("---"),
        mo.md("## ✅ Final Answer"),
        mo.md(example.final_answer),
        mo.md("---"),
        mo.callout(
            f"### 🎯 Why This Matters\n\n{example.connection_to_goal}",
            kind="success"
        ),
        mo.callout(
            f"### 🚀 Try Next\n\n{example.practice_suggestion}",
            kind="info"
        )
    ])
```

Part 5: Deployment & Wrap-Up (30 minutes)

Activity 5.1: Deploy to HuggingFace (15 min)

Live Demonstration:

Step 1: Create Space

1. Go to huggingface.co/new-space
2. Name: `personalized-examples-yourname`
3. SDK: Select "Docker"
4. Template: Select "Marimo"
5. Make it Public
6. Create Space

Step 2: Add Files

Upload three files:

`app.py` - Your complete marimo notebook **`requirements.txt`**:

```txt
marimo>=0.9.0
pydantic>=2.0.0
pydantic-ai>=0.0.13
openai>=1.0.0
```

`README.md`:

```markdown
---
title: Personalized Worked Example Generator
emoji: 🎓
colorFrom: blue
colorTo: green
sdk: docker
pinned: false
---

# Personalized Worked Example Generator

AI-powered tool demonstrating Cognitive Load Theory principles.

Built with Marimo, PydanticAI, and GPT-4o.
```

Step 3: Add API Key

- Settings → Variables and secrets
- Name: `OPENAI_API_KEY`
- Value: Your key
- Save

Step 4: Wait for Build

- Watch build logs
- Usually takes 2-3 minutes
- Test when ready!

Activity 5.2: Testing & Troubleshooting (5 min)

Common Issues:

Issue	Solution
Build fails	Check requirements.txt syntax
"API key not found"	Verify secret name exactly matches `OPENAI_API_KEY`
App won't load	Check marimo version compatibility
Generation fails	Test API key locally first

Activity 5.3: Reflection & Extensions (10 min)

Group Discussion:

What did we learn today?
- Worked example effect from CLT
- How personalization reduces cognitive load
- Building with PydanticAI and structured outputs
- Deploying interactive demos

What could you add?
[Collect ideas]

Possible extensions:
1. More domains (economics, chemistry, history)
2. More concepts per domain
3. Images and diagrams in examples
4. Sequential learning paths
5. Progress tracking
6. Export to PDF
7. Multi-language support

Share Resources:

- Workshop GitHub repo: [Your link]
- CLT literature: NSW CESE (2017)
- PydanticAI docs: ai.pydantic.dev
- Marimo docs: marimo.io

Tool Specification

Complete File Structure

```
personalized-examples/
├── app.py                # Main application
├── requirements.txt      # Dependencies
├── README.md             # Documentation
└── examples/             # Sample outputs (optional)
    ├── programming.md
    ├── health_sciences.md
    └── agronomy.md
```

File 1: `requirements.txt`

```txt
marimo>=0.9.0
pydantic>=2.0.0
pydantic-ai>=0.0.13
openai>=1.0.0
```

File 2: `app.py` **(Complete Implementation)**

See the complete implementation in the next section. This is the full working marimo notebook that implements all the features discussed in the workshop.

Key components:

- Data models (LearnerProfile, PersonalizedWorkedExample)
- Concept library (CONCEPTS dictionary)
- AI agent configuration (example_generator)
- Generation function (generate_personalized_example)
- Interactive UI (marimo widgets and layout)
- Error handling and user feedback

File 3: `README.md`

See the complete README in the deployment guide section below.

Complete app.py Implementation

python

```
"""
Personalized Worked Example Generator
Demonstrates Cognitive Load Theory principles through AI-generated personalized examples.

Based on research:
- Sweller, J. (1988). Cognitive load during problem solving.
- NSW CESE (2017). Cognitive load theory: Research that teachers really need to understand.
"""

import marimo

__generated_with = "0.9.0"
app = marimo.App(width="medium")


@app.cell
def imports():
    import marimo as mo
    from pydantic import BaseModel, Field
    from pydantic_ai import Agent
    from typing import Literal
    import os

    return mo, BaseModel, Field, Agent, Literal, os


@app.cell
def welcome_section(mo):
    mo.md("""
    # 🎓 Personalized Worked Example Generator

    **Learn concepts through examples tailored to YOUR interests!**

    ## Why This Works

    This tool demonstrates principles from **Cognitive Load Theory**:

    ### The Worked Example Effect
    > "Novice learners who are given worked examples to study perform better
    > than learners who are required to solve problems themselves."
    >
    > — NSW Centre for Education Statistics and Evaluation (2017)

    **Why?** Unguided problem-solving overloads working memory. Worked examples
    reduce cognitive load, freeing capacity for learning.
```

```
    ### The Personalization Effect

    Familiar contexts (your hobbies, interests, goals) are easier to process,
    further reducing cognitive load and improving learning.

    ---

    ## How It Works

    1. **Tell us about yourself** - interests, goals, background
    2. **Choose a concept** to learn in your domain
    3. **Get a custom example** woven into your context

    Let's get started! 👇
    """)
    return


@app.cell
def define_models(BaseModel, Field, Literal):
    """Define data models for learner profiles and worked examples"""

    class LearnerProfile(BaseModel):
        """Collect learner information for personalization"""

        name: str = Field(
            description="Learner's first name"
        )

        domain: Literal["programming", "health_sciences", "agronomy"] = Field(
            description="Learning domain"
        )

        specific_interest: str = Field(
            description="Specific interest within domain"
        )

        hobby_or_passion: str = Field(
            description="A hobby or passion they have"
        )

        goal: str = Field(
            description="What they want to achieve"
        )

        background_level: Literal["beginner", "intermediate", "advanced"] = Field(
            description="Their current level in the domain"
```

```
    )


class PersonalizedWorkedExample(BaseModel):
    """A complete worked example tailored to the learner"""

    title: str = Field(
        description="Engaging title that incorporates learner's interest"
    )

    problem_statement: str = Field(
        description="Problem framed in learner's context (3-4 sentences)"
    )

    given_data: str = Field(
        description="Data presented in familiar context (can include code blocks or tables)"
    )

    step_by_step_solution: list[str] = Field(
        description="Clear steps with explanations. Include code or calculations."
    )

    final_answer: str = Field(
        description="The final answer with interpretation relevant to learner's context"
    )

    connection_to_goal: str = Field(
        description="How this example relates to their stated goal (2-3 sentences)"
    )

    practice_suggestion: str = Field(
        description="A similar problem they could try next, using their context"
    )

return LearnerProfile, PersonalizedWorkedExample


@app.cell
def define_concepts():
    """Define concept library for each domain"""

    CONCEPTS = {
        "programming": [
            {
                "name": "For Loops",
                "abstract": "Iterate through a sequence of items",
                "difficulty": "beginner",
```

```
      "typical_use": "Processing lists, repeating actions multiple times"
    },
    {
      "name": "List Comprehensions",
      "abstract": "Create new lists using concise syntax",
      "difficulty": "intermediate",
      "typical_use": "Transform data, filter lists elegantly"
    },
    {
      "name": "Dictionary Methods",
      "abstract": "Access and manipulate key-value pairs",
      "difficulty": "beginner",
      "typical_use": "Store related data, perform fast lookups"
    },
    {
      "name": "Functions with Parameters",
      "abstract": "Create reusable code blocks that accept inputs",
      "difficulty": "beginner",
      "typical_use": "Organize code, avoid repetition"
    },
    {
      "name": "String Formatting",
      "abstract": "Create formatted text output using f-strings",
      "difficulty": "beginner",
      "typical_use": "Display data, create messages dynamically"
    }
  ],
  "health_sciences": [
    {
      "name": "Correlation Analysis",
      "abstract": "Measure strength and direction of relationship between two variables",
      "difficulty": "intermediate",
      "typical_use": "Find relationships in health data, guide research"
    },
    {
      "name": "Mean and Standard Deviation",
      "abstract": "Describe central tendency and variability in data",
      "difficulty": "beginner",
      "typical_use": "Summarize health measurements, describe populations"
    },
    {
      "name": "Independent T-Test",
      "abstract": "Compare means between two independent groups",
      "difficulty": "intermediate",
      "typical_use": "Test intervention effectiveness, compare treatments"
    },
    {
```

```
            "name": "Confidence Intervals",
            "abstract": "Estimate population parameters with uncertainty",
            "difficulty": "intermediate",
            "typical_use": "Interpret research findings, quantify precision"
        },
        {
            "name": "Effect Size (Cohen's d)",
            "abstract": "Measure practical significance of differences",
            "difficulty": "intermediate",
            "typical_use": "Interpret research impact beyond p-values"
        }
    ],
    "agronomy": [
        {
            "name": "Yield Prediction",
            "abstract": "Estimate crop output based on inputs using regression",
            "difficulty": "intermediate",
            "typical_use": "Plan harvest, allocate resources, financial projections"
        },
        {
            "name": "NPK Optimization",
            "abstract": "Calculate optimal fertilizer ratios for maximum benefit",
            "difficulty": "intermediate",
            "typical_use": "Maximize yield while minimizing cost and environmental impact"
        },
        {
            "name": "Growing Degree Days",
            "abstract": "Calculate heat accumulation for crop development",
            "difficulty": "beginner",
            "typical_use": "Predict crop stages, plan field operations"
        },
        {
            "name": "Water Use Efficiency",
            "abstract": "Calculate crop yield per unit of water used",
            "difficulty": "beginner",
            "typical_use": "Optimize irrigation, compare varieties"
        },
        {
            "name": "Cost-Benefit Analysis",
            "abstract": "Compare costs and returns of agricultural interventions",
            "difficulty": "intermediate",
            "typical_use": "Make informed decisions about inputs and practices"
        }
    ]
}

return CONCEPTS,
```

```
@app.cell
def create_agent(Agent, PersonalizedWorkedExample):
    """Create the AI agent for generating personalized examples"""

    example_generator = Agent(
        'openai:gpt-4o',
        result_type=PersonalizedWorkedExample,
        system_prompt="""You are an expert educator who creates highly personalized
        worked examples that connect abstract concepts to learners' lived experiences.

        CRITICAL INSTRUCTIONS:
        1. Weave the learner's interests, hobbies, and goals naturally into the example
        2. Use their name throughout to increase personal connection
        3. Make data and scenarios feel authentic to their context
        4. Keep explanations clear but connect to what they care about
        5. The example should feel like it was written specifically for this person
        6. Match complexity to their level (beginner/intermediate/advanced)
        7. Make the connection to their goal explicit and motivating
        8. Use concrete numbers and realistic data
        9. For programming examples, include actual runnable code with comments
        10. For quantitative examples, show all calculations step by step

        STRUCTURE YOUR EXAMPLES:
        - Start with an engaging title that mentions their interest
        - Frame the problem in their context (use their name and interests)
        - Present data that feels real to their situation
        - Walk through steps clearly with explanations
        - For code: include comments explaining each part
        - For math: show each calculation explicitly
        - Connect the final answer to their goal
        - Suggest a related practice problem in their context

        AVOID:
        - Generic examples with personal details superficially added
        - Forced or artificial connections
        - Too much technical jargon for beginners
        - Abstract variable names (use meaningful names from their context)
        - Skipping steps in solutions

        Remember: This is a WORKED EXAMPLE - a complete solution for the learner
        to study, not a problem for them to solve.
        """
    )

    async def generate_personalized_example(
```

```
        profile: 'LearnerProfile',
        concept: dict
    ) -> PersonalizedWorkedExample:
        """Generate a personalized worked example"""

        prompt = f"""
        Create a worked example for:

        LEARNER PROFILE:
        - Name: {profile.name}
        - Domain: {profile.domain}
        - Specific interest: {profile.specific_interest}
        - Hobby/passion: {profile.hobby_or_passion}
        - Goal: {profile.goal}
        - Level: {profile.background_level}

        CONCEPT TO TEACH:
        - Name: {concept['name']}
        - Abstract description: {concept['abstract']}
        - Difficulty: {concept['difficulty']}
        - Typical use: {concept['typical_use']}

        Create a worked example that teaches this concept using {profile.name}'s
        specific context. The example should feel personal and relevant to their
        goal of "{profile.goal}".

        Make the problem realistic and the data believable for their situation.

        For programming: Include complete, runnable code with explanatory comments.
        For quantitative problems: Show every calculation step explicitly.

        This is a WORKED EXAMPLE - provide the complete solution for them to study.
        """

        result = await example_generator.run(prompt)
        return result.data

    return example_generator, generate_personalized_example


@app.cell
def profile_form_header(mo):
    """Header for profile form"""
    mo.md("---\n## 👤 Step 1: Tell Us About Yourself")
    return
```

```
@app.cell
def profile_inputs(mo):
    """Individual input widgets for profile"""

    name_input = mo.ui.text(
        label="Your first name:",
        placeholder="e.g., Maria",
        full_width=True
    )

    domain_input = mo.ui.dropdown(
        label="Choose your learning domain:",
        options={
            "Programming (Python)": "programming",
            "Health Sciences (Statistics)": "health_sciences",
            "Agronomy (Agricultural Science)": "agronomy"
        },
        value=None,
        full_width=True
    )

    interest_input = mo.ui.text(
        label="Your specific interest in this domain:",
        placeholder="e.g., web development, sports nutrition, coffee farming",
        full_width=True
    )

    hobby_input = mo.ui.text(
        label="A hobby or passion you have:",
        placeholder="e.g., photography, cycling, cooking",
        full_width=True
    )

    goal_input = mo.ui.text(
        label="What you want to achieve:",
        placeholder="e.g., build a portfolio site, improve performance, increase yield",
        full_width=True
    )

    level_input = mo.ui.dropdown(
        label="Your current level:",
        options=["beginner", "intermediate", "advanced"],
        value="beginner",
        full_width=True
    )

    return name_input, domain_input, interest_input, hobby_input, goal_input, level_input
```

```
@app.cell
def display_profile_form(mo, name_input, domain_input, interest_input, hobby_input, goal_input, level_input):
    """Display the profile form"""

    mo.vstack([
        name_input,
        domain_input,
        interest_input,
        hobby_input,
        goal_input,
        level_input
    ])

    return


@app.cell
def check_profile_complete(name_input, domain_input, interest_input, hobby_input, goal_input, level_input):
    """Check if profile is complete"""

    profile_complete = all([
        name_input.value,
        domain_input.value,
        interest_input.value,
        hobby_input.value,
        goal_input.value,
        level_input.value
    ])

    return profile_complete,


@app.cell
def create_profile(profile_complete, LearnerProfile, name_input, domain_input, interest_input, hobby_input, goal_input, leve
    """Create profile object if form is complete"""

    if profile_complete:
        learner_profile = LearnerProfile(
            name=name_input.value,
            domain=domain_input.value,
            specific_interest=interest_input.value,
            hobby_or_passion=hobby_input.value,
            goal=goal_input.value,
            background_level=level_input.value
        )
```

```
    else:
        learner_profile = None

    return learner_profile,


@app.cell
def show_profile_status(mo, profile_complete, learner_profile):
    """Show profile status"""

    if profile_complete:
        mo.callout(
            f"""
            ✅ **Profile Complete!**

            Great, {learner_profile.name}! Now choose a concept below.
            """,
            kind="success"
        )
    else:
        mo.callout(
            "📝 Please fill in all fields above to continue.",
            kind="info"
        )

    return


@app.cell
def concept_selection_header(mo, profile_complete):
    """Header for concept selection"""

    if profile_complete:
        mo.md("---\n## 📚 Step 2: Choose a Concept to Learn")

    return


@app.cell
def concept_selector_widget(mo, profile_complete, learner_profile, CONCEPTS):
    """Create concept selector based on chosen domain"""

    if profile_complete and learner_profile:
        available_concepts = CONCEPTS[learner_profile.domain]

        concept_selector = mo.ui.dropdown(
            label=f"Choose a concept in {learner_profile.domain.replace('_', ' ').title()}:",
```

```
            options={
                f"{c['name']} ({c['difficulty']})": c
                for c in available_concepts
            },
            value=None,
            full_width=True
        )

        concept_selector
    else:
        concept_selector = None

    return concept_selector,


@app.cell
def generate_button_widget(mo, profile_complete, concept_selector):
    """Create generate button"""

    if profile_complete and concept_selector and concept_selector.value:
        mo.md("---")

        generate_button = mo.ui.button(
            label="✨ Generate My Personalized Example",
            kind="success",
            full_width=True
        )

        generate_button
    else:
        generate_button = None

    return generate_button,


@app.cell
async def generate_and_display(mo, generate_button, learner_profile, concept_selector, generate_personalized_example):
    """Generate and display the personalized example"""

    if generate_button and generate_button.value and learner_profile and concept_selector.value:

        # Show loading state
        with mo.status.spinner(title="Creating your personalized example... This may take 30-60 seconds."):
            try:
                example = await generate_personalized_example(
                    profile=learner_profile,
                    concept=concept_selector.value
```

```
            )

            # Display the example
            display_content = mo.vstack([
                mo.md("---"),
                mo.md(f"# {example.title}"),
                mo.md("## 📋 The Problem"),
                mo.md(example.problem_statement),
                mo.md("### Given Data"),
                mo.md(example.given_data),
                mo.md("---"),
                mo.md("## 💡 Step-by-Step Solution"),
                *[mo.md(f"**Step {i}:**\n\n{step}")
                  for i, step in enumerate(example.step_by_step_solution, 1)],
                mo.md("---"),
                mo.md("## ✅ Final Answer"),
                mo.md(example.final_answer),
                mo.md("---"),
                mo.callout(
                    f"### 🎯 Why This Matters for You\n\n{example.connection_to_goal}",
                    kind="success"
                ),
                mo.md("---"),
                mo.callout(
                    f"### 🚀 Try This Next\n\n{example.practice_suggestion}",
                    kind="info"
                ),
            ])

            display_content

        except Exception as e:
            mo.callout(
                f"❌ Error generating example: {str(e)}\n\nPlease check your OpenAI API key.",
                kind="danger"
            )

    return


@app.cell
def footer(mo):
    """Display footer with information"""

    mo.md("""
    ---
```

📖 About This Tool

Cognitive Load Theory Principles

This tool demonstrates research-backed learning principles:

The Worked Example Effect (Sweller, 1988; Cooper & Sweller, 1987)
> "Novice learners who are given worked examples to study perform better on
> subsequent tests than learners who are required to solve the equivalent
> problems themselves."

- **Why?** Unguided problem-solving overloads working memory
- **Result:** Studying worked examples frees cognitive capacity for learning
- **Evidence:** Effect size of 0.52 across multiple studies (Crissman, 2006)

The Personalization Effect (Cordova & Lepper, 1996)
> "Familiar contexts require less cognitive effort to process, reducing
> extraneous cognitive load and improving learning outcomes."

- **Why?** Known contexts don't require working memory to parse
- **Result:** More capacity available for schema construction
- **Benefit:** Increased motivation through personal relevance

Built With

- [Marimo](https://marimo.io) - Reactive Python notebooks
- [PydanticAI](https://ai.pydantic.dev) - Type-safe AI agents
- [OpenAI GPT-4o](https://openai.com) - Language model
- [Pydantic](https://pydantic.dev) - Data validation

🔧 Extend This Tool

Ideas for enhancement:
- Add more domains (economics, chemistry, history, literature)
- Include images and diagrams in examples
- Create sequences of scaffolded examples
- Track learner progress over time
- Export examples to PDF or flashcards
- Add multilingual support
- Integrate with learning management systems

📚 Research References

- Cooper, G., & Sweller, J. (1987). Effects of schema acquisition and rule automation on mathematical problem-solving transfer. *Journal of Educational Psychology*, 79(4), 347-362.

```
    - Cordova, D. I., & Lepper, M. R. (1996). Intrinsic motivation and the process
      of learning: Beneficial effects of contextualization, personalization, and
      choice. *Journal of Educational Psychology*, 88(4), 715.

    - Crissman, J. (2006). *The design and utilization of effective worked examples:
      A meta-analysis* (Doctoral dissertation). University of Nebraska, Lincoln.

    - NSW Centre for Education Statistics and Evaluation (2017). *Cognitive load
      theory: Research that teachers really need to understand*.
      [Link](https://education.nsw.gov.au/about-us/education-data-and-research/cese/publications/literature-reviews/cognitive-l

    - Sweller, J. (1988). Cognitive load during problem solving: Effects on learning.
      *Cognitive Science*, 12(2), 257-285.

    ---

    **Workshop Materials:** [Your GitHub Repository]

    **Contact:** [Your Email]
    """)

    return


if __name__ == "__main__":
    app.run()
```

Deployment Guide

Prerequisites

- Python 3.10 or higher
- OpenAI API key
- HuggingFace account (free)

Local Development

```
bash
```

```
# 1. Clone or download files
git clone [your-repo-url]
cd personalized-examples

# 2. Install dependencies
pip install -r requirements.txt

# 3. Set environment variable
export OPENAI_API_KEY="sk-your-actual-key"

# 4. Run locally
marimo edit app.py

# 5. Open browser to displayed URL
```

HuggingFace Spaces Deployment

Step 1: Create Space

1. Go to https://huggingface.co/new-space
2. Name: `personalized-examples-yourname`
3. SDK: "Docker"
4. Template: "Marimo"
5. Visibility: Public
6. Create Space

Step 2: Upload Files

- `app.py`
- `requirements.txt`
- `README.md`

Step 3: Add Secret

- Settings → Variables and secrets
- Name: `OPENAI_API_KEY`
- Value: Your key
- Save

Step 4: Test

- Wait for build (2-3 minutes)
- Test interface

- Share URL

Extensions & Resources

Extension Ideas

1. **More Domains** (30 min)
   - Add economics, chemistry, history
   - Update domain Literal type
   - Add concepts to CONCEPTS dict
2. **Visual Enhancements** (1 hour)
   - Add matplotlib charts
   - Include code syntax highlighting
   - Generate diagrams
3. **Learning Sequences** (2 hours)
   - Progressive example difficulty
   - Prerequisites tracking
   - Recommended next concepts

Research Resources

- NSW CESE (2017): https://education.nsw.gov.au/...
- Sweller, J. (1988): https://doi.org/10.1207/s15516709cog1202_4
- Cooper & Sweller (1987): https://doi.org/10.1037/0022-0663.79.4.347

Technical Documentation

- Marimo: https://docs.marimo.io
- PydanticAI: https://ai.pydantic.dev
- OpenAI API: https://platform.openai.com/docs

End of Workshop Specification